

Kennesaw State University

Parallel and Distributed Computing

Project – Distributed Computing Topic Presentation

Instructor: Kun Suo

Points Possible: 100

Difficulty: ★★★★★

1. Project purpose

Each group will become the class expert on one distributed-computing topic. Your group will self-study the topic, redesign the supplied instructional material, build or demonstrate a technical example, and teach the topic through a recorded lecture.

Central expectation: Do not merely summarize the existing slides. Teach the topic in your own structure, verify the technical content, add meaningful new material, and demonstrate that every member understands the part they present.

Learning outcomes

1. Explain a distributed-computing concept accurately and at an appropriate technical depth.
2. Connect algorithms and architectures to realistic systems, tradeoffs, and failure modes.
3. Create and test a reproducible code demonstration or technical walkthrough.
4. Collaborate on a coherent lecture while showing clear individual ownership.
5. Use sources and AI tools responsibly, transparently, and critically.

2. Teams and topic assignments

The class has 40+ students, and this project covers 12 topics. The class will use 12 groups, and the instructor will post the group roster and topic assignment in D2L.

Group	Assigned topic	Course theme
1	Distributed mutex	Coordination
2	Distributed election	Coordination
3	Distributed consensus	Coordination
4	Distributed locks	Coordination
5	Centralized structure	Management
6	Decentralized structure	Management
7	MapReduce	Computation
8	Spark	Computation
9	CAP theorem	Storage
10	Distributed storage	Storage
11	Publish and subscribe	Communication
12	Message queue	Communication

3. Fair division of sections and speaking time

Before building the lecture, divide the topic into N meaningful sections or subtopics. If the group has M students, distribute the sections as follows:

- If N is divisible by M , each student presents N/M sections.
- If N is not divisible by M , let $q = \text{floor}(N/M)$ and $r = N \bmod M$. Then r students present $q + 1$ sections and the remaining $M - r$ students present q sections.
- Section count alone is not enough. Balance the assignment by estimated speaking time, technical difficulty, research effort, and demo responsibility.
- A code demo may count as one section when it includes design, implementation, execution, and explanation.

Speaking-time target: For a four-person group, each student should present about 11-15 minutes. For a three-person group, each student should present about 15-20 minutes. Every student must speak for at least 10 minutes, and the complete lecture must remain between 45 and 60 minutes.

Required first slide: responsibility map

Slide 1 must identify the topic, group number, every member, and assigned sections. Highlight ownership with a distinct color for each member, and use the same small color marker or speaker name on that student's slides throughout the deck.

Member	Highlight	Responsible sections	Est. time
Student A	Blue	Motivation; definitions	12 min
Student B	Green	Architecture; core algorithm	14 min
Student C	Orange	Worked example; tradeoffs	12 min
Student D	Purple	Code demo; summary	12 min

4. Required lecture content

Organize the lecture as a coherent lesson rather than a collection of independent mini-presentations. The exact sections may vary by topic, but the final lecture must include:

- Motivation and learning objectives.** Explain the problem and what the class should learn.
- Foundations and terminology.** Define the necessary system model, assumptions, and vocabulary.
- Core mechanism.** Teach the architecture, protocol, algorithm, or theorem step by step.
- Worked example.** Trace a concrete example with messages, states, data flow, or calculations.
- Real-system connection.** Discuss at least one real framework, platform, service, or published system.
- Technical demonstration.** Run and explain code, a simulation, a benchmark, or a reproducible system walkthrough.
- Tradeoffs and failure behavior.** Address performance, scalability, consistency, availability, fault tolerance, or security as applicable.
- Summary and knowledge check.** End with key takeaways for classmates.

5. Existing slides and originality

The supplied slides are reference material, not a finished submission. You may use them to understand the expected scope and to identify important concepts, but your group must produce a meaningfully revised lecture.

- a) At least 50% of the substantive teaching content must be new or substantially redesigned by the group.
- b) No supplied slide may be copied unchanged. A substantially revised slide must improve the explanation, structure, evidence, visualization, example, or technical accuracy - not merely change colors or fonts.
- c) Do not reproduce a long sequence of the supplied deck. Reorganize the narrative around your own learning objectives and teaching plan.
- d) Cite the supplied slides when adapting them, and cite every external diagram, dataset, quotation, benchmark, or nontrivial claim.

6. Code demonstration requirements

The lecture must contain a technical demonstration lasting approximately 10 minutes. The demo should make an important idea visible rather than simply showing source code on screen. You can use 3rd-party code to demo the topic or design the code by your group. Vibe coding through AI is also allowed just to make the demo great.

- State the question the demo answers and connect it to the lecture concept.
- Show the architecture, inputs, commands, and expected behavior before execution.
- Run the demo and interpret the output. Do not treat successful execution as self-explanatory.
- Include at least one meaningful variation, comparison, failure, or edge case when appropriate.
- Provide source code plus a README with prerequisites, setup steps, commands, expected output, and known limitations.
- Prepare a short backup recording or screenshots in case a live run fails during recording.

Topic family	Possible demonstration direction
Coordination	Simulate mutual exclusion, leader election, consensus, or a distributed lock under delay/failure.
Management	Demo or compare how centralized and decentralized control works, discovery, scheduling, or failure handling.
Computation	Run a MapReduce or Spark workload and explain partitioning, shuffle, parallelism, and timing.
Storage	Demonstrate replication, partition behavior, consistency tradeoffs, or a small distributed key-value store.
Communication	Build producer/consumer or publish/subscribe messaging and examine delivery, buffering, and failures.

7. Recorded lecture requirements

- Total duration: 45-60 minutes.
- Every member must narrate and present the sections listed on Slide 1. AI-generated voice or an AI avatar may not replace a student's presentation.
- Use a readable screen resolution of at least 720p, clear audio, and a stable view of slides and demonstrations.
- Edit out extended setup delays, private information, broken attempts, and long silent periods.
- Final submission: Slides and a video lecture made by the whole team